Simulation Execution Control Module (SECM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Architecture Ecosystem: Structured Reality Standards™
Architecture Family: SIMULOS® — Simulation Governance Architecture
Parent Standard: Simulation Execution Standard (SES)
Operational Layer: Simulation Execution Governance Layer
Category: Governance & Enforcement
Subcategory: Simulation Governance
Type: Parent Standard Module
Governed Space: Simulation Execution Control
Version: 1.0
Status: Canonical · Open Module
Effective Date: 19 August 2026
AI-Readable: Yes
Authority: OOF®
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Governed Space

Simulation Execution Control

Canonical Definition

Simulation Execution Control Module (SECM) defines the governance framework through which an instantiated Simulation Execution is initiated, progressed, constrained, synchronized, monitored, intervened in where permitted, paused, resumed, terminated, and completed while preserving the execution conditions necessary to determine what actually occurred during the run.

SECM establishes the active-control layer of Simulation Execution.

Its purpose is to ensure that a simulation does not become an uncontrolled computational process once execution begins and that material runtime changes cannot occur without remaining visible within the execution record.

Operational Role

SECM is the second internal governance space of the Simulation Execution Standard (SES).

The preceding module:

SIM — Simulation Instantiation Module

establishes:

What exact simulation configuration has been instantiated for execution?

SECM asks:

How must that instantiated simulation remain governed while it is actually running?

The governed progression is:

Instantiated Simulation → Execution Authorization → Execution Initiation → Runtime Control → State Monitoring → Intervention / Deviation Handling → Pause / Resume / Termination → Execution Completion Status

---

Module Operational Space

SECM governs: execution authorization, execution initiation, execution start conditions, runtime control, execution sequencing, control-state management, runtime constraints, temporal control, synchronization control, resource control, execution-state transitions, intervention permissions, intervention execution, runtime modification, runtime deviation, anomaly response, pause, resume, restart, cancellation, termination, emergency termination, timeout, dependency failure response, execution boundary enforcement, control loss, control recovery, execution completion, and execution-control traceability.

Execution Authorization

A Simulation Execution may require an explicit authorization before it begins.

Authorization may define: who or what may initiate execution, which Scenario may run, which configuration may be used, which resources may be consumed, which interventions are permitted, which external connections may be active, which control limits apply, and who or what may terminate the run.

Execution Authorization ≠ Simulation Validity

Authorization means the simulation is permitted to run under defined conditions.

It does not establish that: the model is correct, the scenario is sufficient, the output will be valid, or the simulated system is safe.

Execution Initiation

Execution Initiation marks the transition:

Instantiated

↓

Running

A governed initiation may require that: the intended Simulation Instance exists, required dependencies are available, applicable inputs are loaded, runtime conditions are satisfied, control mechanisms are active, and authorization is confirmed.

Start Condition

A Simulation Start Condition establishes the requirements that must exist before execution may begin.

Possible start conditions may include: environment ready, input set available, scenario loaded, parameter configuration applied, required services active, simulation clock synchronized, required hardware connected, required participant ready, or execution authority confirmed.

No Unverified Start Principle

SIMULOS® establishes:

A simulation should not be assumed to have started under its intended configuration merely because the execution engine reports that a run began.

Material start conditions should be verifiable where necessary.

Execution State

A Simulation Execution should maintain an identifiable current state.

Possible states may include:

Prepared

Initialized

Running

Paused

Waiting

Degraded

Interrupted

Restarting

Terminating

Completed

Failed

or:

Execution State Unknown

depending upon implementation.

Execution State Transition

Material changes between execution states should remain identifiable.

Conceptually:

Initialized → Running → Paused → Running → Completed

or:

Running → Degraded → Failed

The execution history should preserve the actual path.

Runtime Control

Runtime Control governs how the active Simulation Execution remains aligned with the conditions under which it was instantiated.

This may include: state progression, event sequencing, clock progression, resource allocation, scenario progression, environmental updates, external inputs, actor activation, dependency use, and termination criteria.

Execution Control ≠ Outcome Control

SIMULOS® establishes:

Execution Control must govern how the simulation runs, not manipulate the simulation merely to force a preferred output.

Unexpected results should remain observable.

Unfavorable system behavior must not become a reason to silently alter the run.

Runtime Constraint

A Runtime Constraint defines a condition that must remain satisfied during execution.

Examples may include: allowed resource consumption, allowed scenario bounds, permitted external interfaces, maximum execution duration, parameter locks, environmental limits, intervention permissions, or synchronization requirements.

Constraint Violation

A Constraint Violation occurs when actual execution moves outside a materially relevant runtime condition.

The response may include:

Continue with Record

Pause

Restrict

Reconfigure

Restart

Terminate

or:

Escalate

depending upon the Simulation Purpose.

Execution Boundary

The Execution Boundary defines what the active simulation is permitted to do.

It may govern: accessible resources, connected systems, external actions, scenario range, execution duration, control authority, or other material runtime limits.

Boundary Enforcement

A simulation should not silently operate outside the execution boundary merely because the underlying technology permits it.

Execution Sequence

Some simulations require governed sequencing.

For example:

Initialize

↓

Activate Environment

↓

Activate Agents

↓

Introduce Scenario Event

↓

Observe Response

↓

Terminate

Changing sequence may materially change the simulation.

SECM therefore governs sequence where order matters.

No Sequence Substitution Principle

SIMULOS® establishes:

A materially different execution sequence should not be represented as the same controlled execution merely because the same components were eventually activated.

Temporal Control

Simulation execution may rely upon: simulation time, wall-clock time, event time, mission time, synchronized system time, or accelerated / slowed simulation time.

SECM governs the execution-control side of these timing conditions.

Clock Control

Where the simulation clock materially affects behavior, relevant clock state should remain controlled.

This may include: clock start, clock rate, time acceleration, time pause, synchronization, reset, and clock discontinuities.

Synchronization Control

Distributed simulations may require multiple: agents, models, services, hardware systems, sensors, or environments

to remain synchronized.

Synchronization failure may create behavior that reflects infrastructure error rather than the intended scenario.

Synchronization Deviation

A material synchronization deviation should remain identifiable.

Possible treatment may include: tolerance-based continuation, pause, resynchronization, partial restart, run invalidation, or termination.

Resource Control

Execution may require controlled access to: compute, memory, storage, network bandwidth, energy, hardware, external services, or human participants.

Resource constraints may materially alter simulation behavior.

Resource Exhaustion

A simulation may terminate or degrade because computational or infrastructural resources are exhausted.

This should remain distinguishable from:

failure of the simulated system itself.

No Infrastructure-Failure-as-System-Failure Principle

SIMULOS® establishes:

Failure of the simulation infrastructure must not automatically be interpreted as failure of the Simulation Object.

Dependency Control

Simulation execution may depend upon: APIs, external models, databases, hardware, external services, or distributed infrastructure.

Dependency state should remain visible where changes can materially affect execution.

Dependency Failure

Possible responses to dependency failure may include:

Fallback

Pause

Substitute

Continue Degraded

Restart

or:

Terminate.

The selected response should remain traceable.

Runtime Intervention

A Runtime Intervention is a deliberate action that materially changes an active Simulation Execution.

Examples include: parameter modification, scenario injection, human command, failure injection, environment modification, control transfer, agent override, pause, or forced state transition.

Intervention Authority

Where interventions are allowed, SECM should identify who or what may perform them.

Possible intervention authorities include:

Human Operator

Simulation Controller

Automated Control Logic

AI Controller

Autonomous Simulation Agent

or another governed execution actor.

No Invisible Intervention Principle

A material runtime intervention must remain visible in the execution history.

Planned Intervention

Some interventions are part of Scenario Design.

Examples include: injecting a failure at time T, reducing communication availability, triggering handover, or introducing an environmental event.

Such interventions remain governed actions even where planned.

Unplanned Intervention

An intervention may occur unexpectedly because: simulation instability develops, infrastructure fails, safety boundaries are crossed, an operator acts, or an execution anomaly requires control.

Unplanned interventions should be distinguished from planned scenario events.

Runtime Modification

A simulation may permit certain configuration changes during execution.

Material modifications may include: parameter adjustment, model switching, environment changes, agent activation, dependency replacement, or runtime configuration change.

Where permitted, the modification should remain identifiable.

No Silent Configuration Mutation Principle

SIMULOS® establishes:

A materially changed runtime configuration must not continue to be represented as if the original execution configuration remained unchanged.

Execution Deviation

An Execution Deviation is a material difference between:

intended controlled execution

and:

actual runtime behavior or condition.

Possible deviations include: wrong sequence, unexpected delay, missing dependency, incorrect configuration, intervention, resource exhaustion, synchronization failure, runtime exception, timing deviation, or control-state mismatch.

Deviation Classification

A deviation may be classified as:

Minor

Material

Critical

Execution-Limiting

Execution-Invalidating

or:

Undetermined

depending upon the simulation context. Deviation ≠ Automatic Failure

A deviation does not necessarily invalidate a simulation.

The module requires the deviation to remain explicit so its consequence can later be assessed.

Execution Anomaly

An Execution Anomaly is an unexpected runtime event or state that differs materially from expected execution behavior.

An anomaly may originate from: the Simulation Object, the Simulation Model, the environment, infrastructure, external dependencies, configuration, or control logic.

SECM does not determine the cause automatically.

It ensures the event remains governable.

Control Response

A material anomaly may trigger a controlled response such as:

Observe

Continue

Restrict

Pause

Escalate

Recover

Restart

or:

Terminate. Pause

A Simulation Execution may enter a controlled Paused State.

Pause should preserve sufficient execution state to determine: when the pause occurred, why it occurred, what state existed, what changed while paused, and whether resume remains comparable to uninterrupted execution.

Pause ≠ No Change

The simulation may be paused while: external services continue changing, hardware state changes, humans act, external clocks advance, or dependencies change.

Therefore the assumption that a paused simulation preserves identical execution conditions may require verification. Resume

Resume returns a paused Simulation Execution to active execution.

Material changes occurring during the pause should remain visible. Restart

A Restart begins execution again after: interruption, failure, modification, or control decision.

Restart may occur:

from initial state

or:

from checkpoint.

These are materially different execution paths. Checkpoint

A simulation may preserve a checkpoint from which execution can later resume.

A checkpoint should remain connected to: Execution ID, execution state, simulation clock, configuration, relevant memory/state, environment, and applicable dependencies.

Checkpoint ≠ Original Execution

Restart from checkpoint may preserve continuity, but it should not automatically be represented as identical to uninterrupted execution.

Execution Interruption

An interruption occurs when active execution stops before intended completion.

Possible causes include: infrastructure failure, external dependency failure, operator intervention, resource exhaustion, software failure, or emergency control.

Interruption Status

An interrupted run may become:

Resumable

Restart Required

Terminated

Failed

or:

Status Undetermined.

Execution Termination

Termination deliberately ends an execution.

Termination may be:

Normal

Conditional

Early

Failure-Based

Safety-Based

Resource-Based

Operator-Initiated

Automatically Triggered

or:

Emergency

where relevant.

Termination Criteria

Termination criteria should be established where materially relevant.

Examples include: scenario complete, target state reached, maximum duration reached, failure state reached, invalid simulation region entered, resource limit exceeded, safety boundary crossed, or information objective achieved.

Early Termination

An execution may terminate before planned completion because the simulation has already reached a state that satisfies or invalidates the execution purpose.

Early termination should remain explicit.

Emergency Termination

Emergency Termination may be required when continuing execution could: damage hardware, affect external systems, violate resource constraints, create unsafe human interaction, or move beyond authorized simulation boundaries.

Timeout

A Timeout occurs when execution exceeds a predefined temporal or processing condition.

Timeout should remain distinguishable from: simulated-system failure, execution infrastructure failure, or intentional termination.

Execution Completion

Execution Completion occurs when the Simulation Execution reaches an accepted terminal state.

Completion may be:

Complete

Conditionally Complete

Partially Complete

Incomplete

Failed

or:

Completion Undetermined. Completion ≠ Positive Result

SIMULOS® establishes:

A fully completed simulation may show complete system failure, while an incomplete simulation may appear to show a positive result.

Execution completion and simulation outcome must remain distinct.

Control Loss

A Simulation Execution Control Loss occurs where the execution can no longer be governed according to its required runtime controls.

Examples include: inability to pause, inability to terminate, lost synchronization, uncontrolled parameter change, uncontrolled external interaction, controller failure, or execution-state uncertainty.

Control Loss Response

Where control loss is material, the execution may require:

Containment

Emergency Termination

Isolation

Restart

or:

Execution Invalidity Assessment.

Control Recovery

Where control is restored, the execution should preserve: when control was lost, how long control was absent, what occurred during the gap, how control was recovered, and whether execution continuity remains acceptable.

Autonomous Execution Control

Simulation campaigns may increasingly be controlled by autonomous orchestration systems.

An autonomous controller may: launch runs, select scenarios, modify configurations, detect anomalies, stop executions, allocate resources, or generate follow-up runs.

SECM remains applicable.

The controller's execution authority and actions should remain governable.

No Autonomous Control Opacity Principle

SIMULOS® establishes:

Autonomous execution control must not remove visibility into why materially relevant runtime actions occurred.

Adaptive Execution Control

Some simulations adapt during execution.

The simulation may:

Observe Behavior

↓

Change Runtime Condition

↓

Continue Execution

↓

Observe New Behavior

This may be legitimate.

However, the adaptive path should remain explicit.

Execution-Control Traceability

SECM should support reconstruction of:

Who / What Controlled the Run

↓

Which Controls Applied

↓

What Execution State Existed

↓

What Interventions Occurred

↓

What Deviations Occurred

↓

How the Run Ended Simulation Campaign Control

For large simulation campaigns, execution control may occur at two levels:

Campaign-Level Control

and:

Run-Level Control.

Campaign-level rules may establish: permitted scenarios, run counts, resource allocations, stopping criteria, failure handling, retry logic, and exception treatment.

Individual runs may still require their own execution-state history.

Retry Governance

Automated campaigns may retry failed runs.

A retry should remain distinguishable from the original execution.

Conceptually:

Run 001 — Failed

↓

Retry 001A

Rather than:

Run 001 — Successful

where the first failure disappears.

No Failure Erasure by Retry Principle

SIMULOS® establishes:

A successful retry must not erase the existence of the execution failure that triggered it.

Execution Selection Bias

Large campaigns may automatically discard runs classified as: failed, anomalous, incomplete, or outlying.

SECM requires material execution-control exclusions to remain visible.

No Success-Only Preservation Principle

Execution governance must not preserve only completed or desirable runs where failed, interrupted, anomalous, or deviating executions are materially relevant to understanding the simulation.

Space and Mission Execution Control

Mission simulations may include: spacecraft dynamics, communication delay, autonomous response, hardware-in-the-loop systems, ground-control simulation, navigation systems, power simulation, and environmental models.

A runtime deviation may originate from:

the simulated spacecraft

or:

the simulation infrastructure.

SECM preserves this distinction.

A mission run showing loss of navigation cannot automatically be interpreted as spacecraft navigation failure if synchronization loss inside the simulator caused the event.

Autonomous Systems

For autonomous systems, execution control may need to preserve: autonomy mode, human override, permission state, fallback state, communication state, external-agent interaction, and runtime authority transitions.

This becomes particularly important where the simulation itself allows dynamic changes in authority.

Industrial Systems

A digital-twin or industrial simulation may interact with: real hardware, production data, live sensor feeds, or control systems.

SECM provides the control layer needed to prevent simulation execution from

becoming an unbounded interaction with operational infrastructure.

Minimum Implementation Framework

1. Establish Execution Authorization

Identify who or what may initiate, control, intervene in, and terminate the Simulation Execution. 2. Verify Start Conditions

Confirm the materially required configuration, dependencies, controls, and runtime conditions before execution begins. 3. Initiate the Execution

Create the active execution state and preserve the execution start record. 4. Maintain Runtime Control

Govern: sequence, timing, synchronization, resources, dependencies, state transitions, and applicable runtime boundaries.

5. Govern Interventions

Record and classify planned and unplanned runtime interventions. 6. Govern Deviations and Anomalies

Identify material differences between intended and actual execution. 7. Govern Pause, Resume, Restart, and Retry

Preserve execution continuity and distinguish interrupted or repeated execution paths. 8. Govern Termination

Apply explicit termination criteria and preserve the reason and state at termination. 9. Determine Completion Status

Classify execution as: Complete, Conditionally Complete, Partially Complete, Incomplete, Failed, or Undetermined.

10. Preserve Execution-Control Traceability

Maintain the runtime control history required for subsequent observation, repetition, and execution documentation.

Governance Outputs

SECM may produce: Simulation Execution Authorization Record, Simulation Start Condition Record, Execution Initiation Record, Execution State Record, Execution State Transition Record, Runtime Control Record, Runtime Constraint Record, Execution Boundary Record, Execution Sequence Record, Simulation Clock Control Record, Synchronization Control Record, Synchronization Deviation Record, Resource Control Record, Resource Exhaustion Record, Dependency Control Record, Dependency Failure Record, Runtime Intervention Record, Intervention Authority Record, Planned Intervention Record, Unplanned Intervention Record, Runtime Modification Record, Execution Deviation Record, Execution Deviation Classification, Execution Anomaly Record, Control Response Record, Pause Record, Resume Record, Restart Record, Checkpoint Record, Execution Interruption Record, Execution Termination Record, Emergency Termination Record, Timeout Record, Control Loss Record, Control Recovery Record, Retry Record, Execution Completion Record, Execution Completion

Status, Campaign Control Record, Run-Level Exception Record, and Simulation Execution Control Traceability Record.

Use Case 1 — Autonomous Vehicle Simulation Campaign

Scenario

A large autonomous-driving simulation campaign runs thousands of scenarios.

One execution shows unexpected failure of collision avoidance.

The run was automatically retried and the second execution completed successfully. Application

SECM preserves:

Original Run — Failure

Runtime Conditions

Execution Deviation

Retry Trigger

Retry Execution

Differences Between Runs

rather than replacing the failed run with the successful retry. Result

Engineers can determine whether the original behavior came from: the autonomous system, simulation infrastructure, timing, synchronization, runtime configuration, or another execution condition.

The failed execution remains part of the governed simulation history. Use Case 2 — Spacecraft Hardware-in-the-Loop Simulation

Scenario

A spacecraft autonomy simulation is connected to flight-like hardware and communication subsystems.

During an autonomous recovery scenario, synchronization between the simulation environment and hardware begins to drift. Application

SECM identifies a material Synchronization Deviation.

Execution enters:

Paused

rather than silently continuing.

The synchronization state is restored and the execution is resumed from an

identified checkpoint. Result

The mission team can distinguish:

behavior of the spacecraft autonomy logic

from:

behavior produced by simulation-infrastructure timing error.

The resumed execution remains traceable to the synchronization deviation and checkpoint rather than being represented as an uninterrupted nominal run.

Architectural Position

Simulation Execution Control Module (SECM) is the second internal governance space of the Simulation Execution Standard (SES).

The complete SES progression is:

Simulation Instantiation

↓

Simulation Execution Control

↓

Simulation Observation

↓

Simulation Repetition

↓

Simulation Execution Documentation

SIM establishes:

What exactly has been instantiated?

SECM establishes:

How does that instantiated simulation remain controlled while running?

The next module, SOM — Simulation Observation Module, governs what materially relevant states, events, transitions, anomalies, and runtime behavior must be observed and preserved during execution.

Foundational Principle

A simulation execution remains governable only while the conditions controlling how it starts, progresses, changes, pauses, resumes, deviates, and

terminates remain explicit and traceable.

Canonical Closing Statement

Simulation Execution Control Module establishes the runtime governance layer of the Simulation Execution Standard by governing authorization, start conditions, execution states, sequencing, timing, synchronization, resource and dependency control, interventions, deviations, anomalies, pause and resume, restart, retry, termination, control loss, recovery, and completion. SECM ensures that active simulation execution cannot silently diverge from its instantiated basis, that infrastructure failures remain distinguishable from simulated-system failures, that retries do not erase failed runs, and that every materially relevant runtime control event remains reconstructable as part of the governed Simulation Execution history.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

https://originopenfoundation.org/